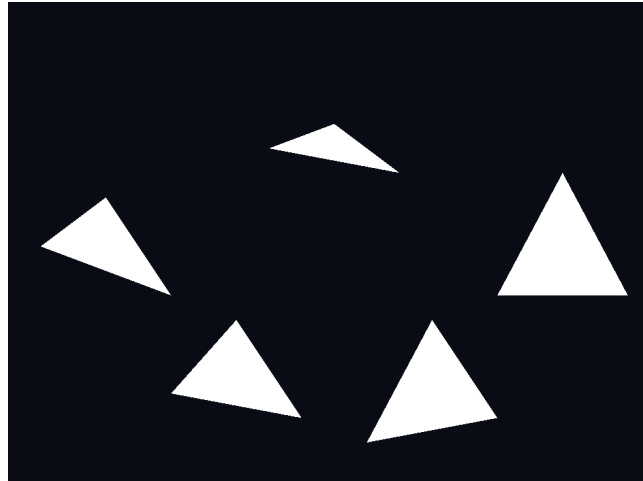


Visual Computing - Assignment 1

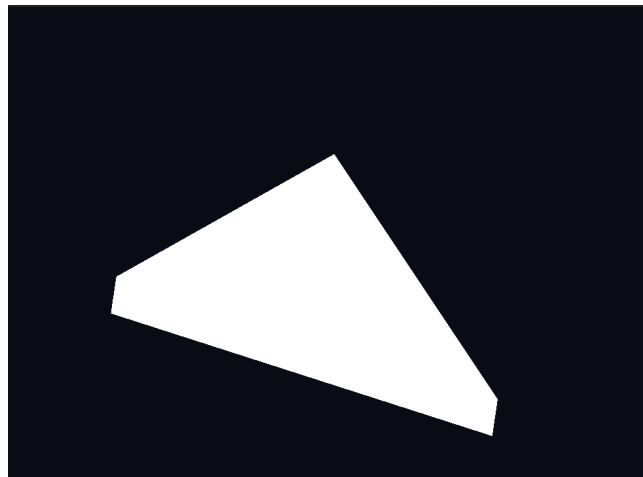
Pierre CHHIENG

Task 1



Task 2

a)



i) What is the name of this phenomenon?

This phenomenon is clipping.

ii) When does it occur?

Clipping happens whenever part of a primitive lies outside the view volume. In OpenGL, this volume is the normalized cube that spans from -1 to 1 along the x , y , and z axes.

iii) What is its purpose?

The purpose of clipping is to avoid unnecessary computations, and guarantees that the rendered image stays within the bounds of the display window.

b)



Index : 0, 1, 2



Index : 0, 2, 1

i) What happens?

When I swapped the order of two vertices in the index buffer, the triangle disappeared from the screen.

ii) Why does it happen?

When the order of the vertices is changed, the triangle is turned around, and OpenGL sees it as the back side, so it is not rendered. This reduces computation and makes rendering faster.

iii) What is the condition under which this effect occurs? Determine a rule.

OpenGL decides whether a triangle is facing the camera or turned away based on the order of its vertices. If the vertices are ordered counter-clockwise, the triangle is seen as front-facing and gets drawn. If they are ordered clockwise, the triangle is seen as back-facing and gets removed.

c)

i) Why does the depth buffer need to be reset each frame?

The depth buffer keeps track of how far each pixel is from the camera. If it is not reset every frame, old "depth information" can interfere with the new frame.

ii) In which situation can the Fragment Shader be executed multiple times for the same pixel? (Assume we do not use multisampling.)

The fragment shader runs once for each triangle fragment that covers a pixel. If several triangles overlap at the same pixel, the shader will run multiple times. The depth test then determines which fragment is actually visible.

iii) What are the two most commonly used types of Shaders? What are the responsibilities of each of them?

- Vertex Shader: Processes each vertex and transforms the vertex from its local coordinates into the correct position in the scene. This includes applying translations, rotations, scalings, and projecting the geometry into the camera's perspective.
- Fragment Shader : Runs for every pixel that a triangle covers and decides the final color of that pixel. It can handle textures, lighting, and other visual effects.

iv) Why is it common to use an index buffer to specify which vertices should be connected into triangles, as opposed to relying on the order in which the vertices are specified in the vertex buffer(s)?

Using an index buffer allows the GPU to reuse the same vertex multiple times instead of duplicating it. This reduces memory usage and ensures that connected triangles form a consistent surface.

v) While the last input of `gl::VertexAttribPointer()` is a pointer, we usually pass in a null pointer. Describe a situation in which you would pass a non-zero value into this function.

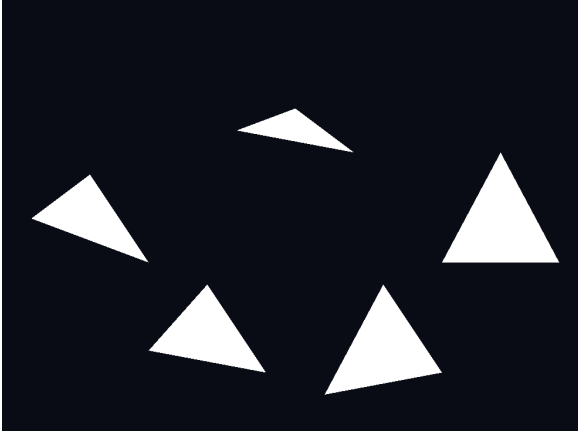
A non-zero value is used when attributes are stored at an offset inside a buffer. For example, if positions and texture coordinates are stored together, the texture coordinates start after the position data, so the pointer specifies the offset to the correct data.

d)

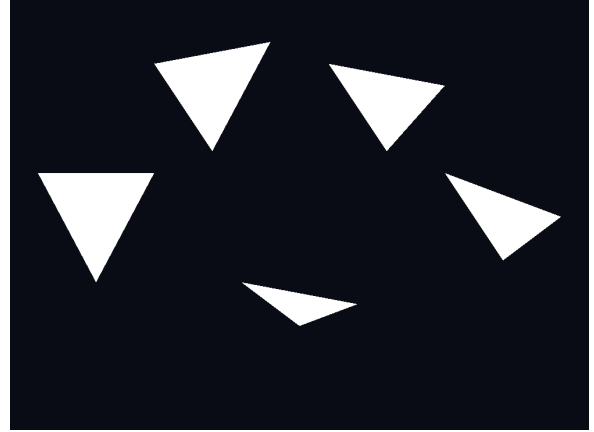
i)

To flip the entire scene, I created a scaling matrix with factors $(-1.0, -1.0, 1.0)$. This matrix inverts the x and y coordinates, which results in both a horizontal and a vertical mirror. The matrix was passed as a uniform (mirrorMatrix) to the vertex shader, and then applied to the vertex positions using:

```
gl_Position = mirrorMatrix * vec4(position, 1.0);
```



Original image



Mirrored image

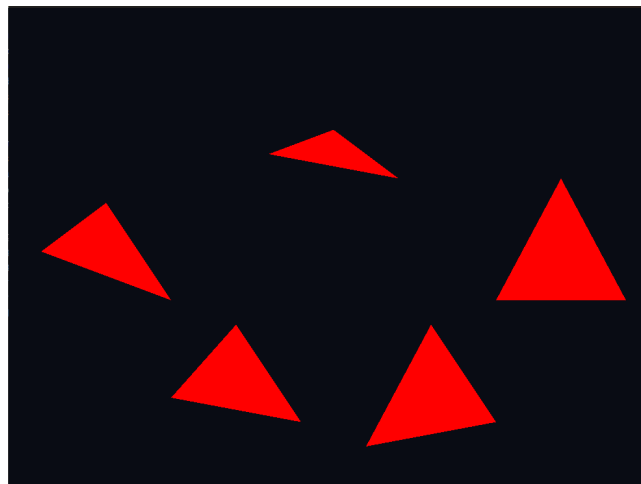
ii)

In the fragment shader, I hardcoded the output color. Originally, the shader set the color to white :

```
color = vec4(1.0, 1.0, 1.0, 1.0);
```

Then, I modified this line to use different values, for example:

```
color = vec4(1.0, 0.0, 0.0, 1.0);
```



Bonus

Checkboard

